Indian Institute of Technology Jodhpur

Department of Computer Science & Engineering

Assignment 5

LoRA Fine-tuning & Adversarial Robustness

Course : MLOps

Roll No. : B23CS1061

Name : Sahilpreet Singh

Deadline : 03 April 2026

W&B Project : `final-ops-ass-5`

HuggingFace : `sps1001/Assignment-5-ops`

GitHub : `Assignment-5 branch`

All experiments run inside Docker containers using Python & PyTorch.

0 Contents

1 Question 1 — ViT-S + LoRA Fine-tuning on CIFAR-100 **2**
- 1.1 Overview . . . 2
- 1.2 Table 1 — Baseline (Head-Only, No LoRA) — 10 Epochs . . . 2
- 1.3 LoRA Epoch-wise Tables — All 9 Configurations . . . 3
- 1.4 Testing Results — CIFAR-100 Test Set . . . 7
- 1.5 Q1 Figures . . . 8
- 1.6 Analysis — Q1 . . . 9

2 Question 2 — Adversarial Attacks on CIFAR-10 (IBM ART) **10**
- 2.1 Overview . . . 10
- 2.2 ResNet-18 Training Dynamics (CIFAR-10) . . . 10
- 2.3 Q2(i) — FGSM Attack: Scratch vs IBM ART (Table 3) . . . 10
- 2.4 Q2(ii) — Adversarial Detectors (ResNet-34) . . . 11
- 2.5 Analysis — Q2 . . . 13

3 Conclusions **14**

4 Links & Reproducibility **14**

1 Question 1 — ViT-S + LoRA Fine-tuning on CIFAR-100

1.1 Overview

A Vision Transformer Small (`WinKawaks/vit-small-patch16-224`), pre-trained on ImageNet, was fine-tuned on CIFAR-100 under two regimes:

1. **Baseline (head-only):** Only the 100-class classification head (38,500 parameters) is updated; the transformer backbone is frozen.
2. **LoRA (Q,K,V):** Low-Rank Adaptation matrices are injected into the Query, Key, and Value projection weights of every attention layer, in addition to a trainable classification head. Experiments span rank $\in \{2, 4, 8\}$ and $\alpha \in \{2, 4, 8\}$ with dropout $= 0.1$.

Optuna was used to search LoRA hyperparameters only; the best configuration found was rank $= 4$, $\alpha = 8$, dropout $= 0.1$.

1.2 Table 1 — Baseline (Head-Only, No LoRA) — 10 Epochs

Table 1: Epoch-wise train/val loss and accuracy for the baseline run (head-only, no LoRA).

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	2.2699	1.2275	0.5286	0.7146
2	1.0102	0.9497	0.7476	0.7527
3	0.8284	0.8504	0.7780	0.7648
4	0.7426	0.8067	0.7953	0.7727
5	0.6882	0.7745	0.8074	0.7807
6	0.6470	0.7567	0.8175	0.7836
7	0.6151	0.7461	0.8265	0.7824
8	0.5903	0.7298	0.8318	0.7881
9	0.5684	0.7269	0.8365	0.7889
10	0.5491	0.7232	0.8418	0.7910

1.3 LoRA Epoch-wise Tables — All 9 Configurations

Experiment 1 — rank=2, alpha=2, dropout=0.1

Table 2: LoRA (Q,K,V) — rank=2 — alpha=2 — dropout=0.1

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	4.7354	4.4796	0.0153	0.0334
2	4.0179	3.5911	0.0921	0.1570
3	3.3160	3.1183	0.2200	0.2648
4	2.9174	2.7963	0.3138	0.3496
5	2.6365	2.5746	0.3869	0.4027
6	2.4439	2.4207	0.4347	0.4410
7	2.3018	2.3019	0.4723	0.4729
8	2.1923	2.2082	0.4981	0.4955
9	2.1046	2.1375	0.5183	0.5082
10	2.0304	2.0634	0.5370	0.5285

Experiment 2 — rank=2, alpha=4, dropout=0.1

Table 3: LoRA (Q,K,V) — rank=2 — alpha=4 — dropout=0.1

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	4.6525	4.2211	0.0231	0.0592
2	3.6105	3.1882	0.1575	0.2465
3	2.9045	2.7198	0.3239	0.3676
4	2.5306	2.4474	0.4155	0.4449
5	2.2953	2.2711	0.4740	0.4830
6	2.1376	2.1363	0.5136	0.5191
7	2.0184	2.0308	0.5451	0.5404
8	1.9247	1.9502	0.5703	0.5549
9	1.8483	1.8953	0.5891	0.5721
10	1.7849	1.8360	0.6024	0.5840

Experiment 3 — rank=2, alpha=8, dropout=0.1

Table 4: LoRA (Q,K,V) — rank=2 — alpha=8 — dropout=0.1

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	4.5924	3.9906	0.0253	0.0773
2	3.2575	2.8078	0.2333	0.3385
3	2.5423	2.3845	0.4131	0.4480
4	2.2079	2.1613	0.4981	0.5127
5	2.0116	1.9955	0.5494	0.5486
6	1.8792	1.8904	0.5834	0.5828
7	1.7733	1.8019	0.6116	0.5971
8	1.6907	1.7222	0.6313	0.6168
9	1.6226	1.6656	0.6479	0.6307
10	1.5617	1.6210	0.6631	0.6396

Experiment 4 — rank=4, alpha=2, dropout=0.1

Table 5: LoRA (Q,K,V) — rank=4 — alpha=2 — dropout=0.1

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	4.7559	4.5487	0.0126	0.0230
2	4.0346	3.4724	0.0874	0.1836
3	3.0866	2.8226	0.2794	0.3465
4	2.6014	2.4829	0.3988	0.4316
5	2.3286	2.2729	0.4695	0.4830
6	2.1415	2.1186	0.5144	0.5146
7	1.9996	2.0047	0.5509	0.5365
8	1.8812	1.8948	0.5783	0.5707
9	1.7827	1.8133	0.6026	0.5826
10	1.6981	1.7458	0.6244	0.6010

Experiment 5 — rank=4, alpha=4, dropout=0.1

Table 6: LoRA (Q,K,V) — rank=4 — alpha=4 — dropout=0.1

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	4.6521	4.1585	0.0201	0.0604
2	3.3601	2.8422	0.2190	0.3371
3	2.5345	2.3592	0.4223	0.4576
4	2.1728	2.0974	0.5103	0.5236
5	1.9469	1.9147	0.5671	0.5652
6	1.7813	1.7774	0.6047	0.5992
7	1.6519	1.6877	0.6368	0.6125
8	1.5452	1.5832	0.6634	0.6428
9	1.4598	1.5150	0.6806	0.6615
10	1.3839	1.4415	0.6996	0.6762

Experiment 6 — rank=4, alpha=8, dropout=0.1 (Optuna Best)

Table 7: LoRA (Q,K,V) — rank=4 — alpha=8 — dropout=0.1 [**Optuna Best**]

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	4.3584	3.3715	0.0579	0.2051
2	2.7394	2.3815	0.3698	0.4557
3	2.1295	1.9997	0.5242	0.5424
4	1.8244	1.7710	0.5980	0.5951
5	1.6220	1.6102	0.6463	0.6328
6	1.4696	1.4730	0.6825	0.6682
7	1.3506	1.3854	0.7118	0.6910
8	1.2554	1.3009	0.7345	0.7141
9	1.1730	1.2338	0.7544	0.7318
10	1.1042	1.1837	0.7700	0.7418

Experiment 7 — rank=8, alpha=2, dropout=0.1

Table 8: LoRA (Q,K,V) — rank=8 — alpha=2 — dropout=0.1

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	4.7319	4.4035	0.0166	0.0416
2	3.6919	3.1142	0.1536	0.2650
3	2.7799	2.5811	0.3590	0.4082
4	2.3708	2.2701	0.4614	0.4879
5	2.1020	2.0446	0.5286	0.5340
6	1.8970	1.8662	0.5776	0.5744
7	1.7333	1.7272	0.6172	0.6090
8	1.5994	1.6155	0.6506	0.6273
9	1.4865	1.5103	0.6734	0.6604
10	1.3896	1.4240	0.6957	0.6791

Experiment 8 — rank=8, alpha=4, dropout=0.1

Table 9: LoRA (Q,K,V) — rank=8 — alpha=4 — dropout=0.1

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	4.5421	3.7459	0.0359	0.1254
2	2.9974	2.5641	0.3051	0.4076
3	2.2805	2.1180	0.4820	0.5139
4	1.9178	1.8350	0.5708	0.5799
5	1.6766	1.6331	0.6280	0.6303
6	1.4916	1.4777	0.6719	0.6664
7	1.3459	1.3523	0.7090	0.7002
8	1.2292	1.2522	0.7335	0.7127
9	1.1330	1.1664	0.7558	0.7312
10	1.0505	1.1050	0.7756	0.7486

Experiment 9 — rank=8, alpha=8, dropout=0.1

Table 10: LoRA (Q,K,V) — rank=8 — alpha=8 — dropout=0.1

Epoch	Train Loss	Val Loss	Train Acc.	Val Acc.
1	4.2160	3.0433	0.0871	0.2859
2	2.4636	2.1178	0.4354	0.5227
3	1.8565	1.7173	0.5863	0.6090
4	1.5213	1.4506	0.6668	0.6662
5	1.3004	1.2721	0.7186	0.7061
6	1.1385	1.1342	0.7563	0.7414
7	1.0154	1.0357	0.7832	0.7689
8	0.9188	0.9652	0.8060	0.7750
9	0.8401	0.8958	0.8210	0.7934
10	0.7770	0.8602	0.8363	0.7961

1.4 Testing Results — CIFAR-100 Test Set

Table 11: Overall test accuracy for all LoRA configurations and the baseline on CIFAR-100.

LoRA Layers	Rank	Alpha	Dropout	Test Acc.	Trainable Params
Without LoRA (head-only)	—	—	—	**0.7901**	38,500
With LoRA (Q,K,V)	2	2	0.1	0.5314	55,296
With LoRA (Q,K,V)	2	4	0.1	0.5901	55,296
With LoRA (Q,K,V)	2	8	0.1	0.6387	55,296
With LoRA (Q,K,V)	4	2	0.1	0.6104	110,592
With LoRA (Q,K,V)	4	4	0.1	0.6849	110,592
With LoRA (Q,K,V)	4	8	0.1	**0.7408**	110,592
With LoRA (Q,K,V)	8	2	0.1	0.6869	221,184
With LoRA (Q,K,V)	8	4	0.1	0.7530	221,184
With LoRA (Q,K,V)	8	8	0.1	**0.8046**	221,184

Optuna best: rank=4, α=8, dropout=0.1 $\Rightarrow$ test accuracy **0.7408**
Checkpoint: `q1/outputs/best_q1-lora-r4-a8-d0.1-full.pt`

1.5 Q1 Figures

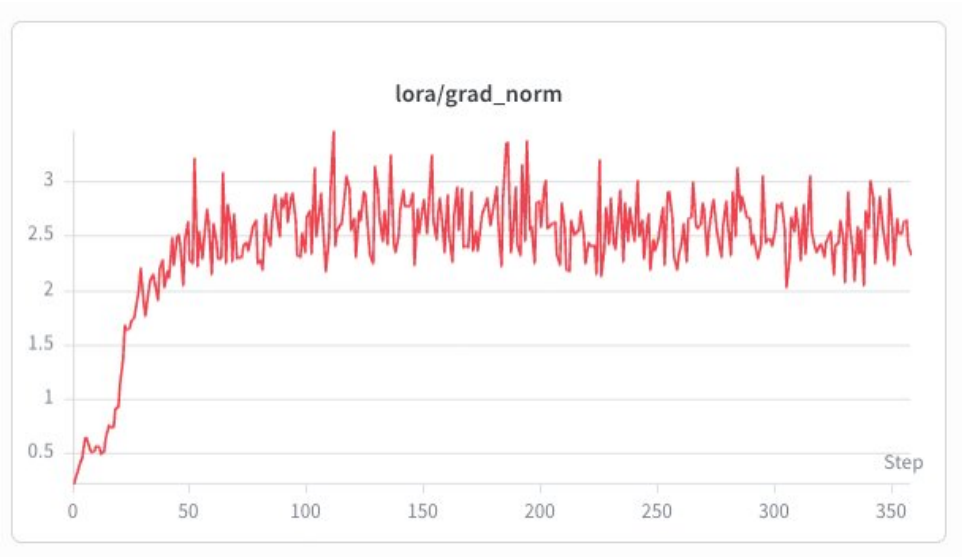


Figure 1: Figure 1.1 — LoRA gradient norm (`lora/grad_norm`) during training of the best LoRA run (rank=4, α=8). The gradient norm ramps up from near zero and stabilises around 2.5, indicating healthy and sustained learning in the LoRA adapters.

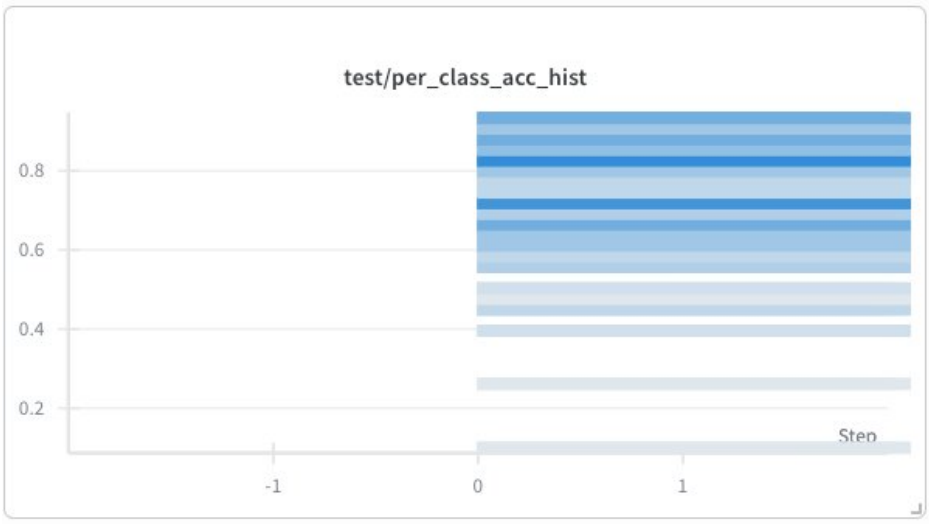


Figure 2: Figure 1.2 — Per-class test accuracy histogram for the **best LoRA** model (rank=8, α=8). Class accuracies span roughly 0.20–0.90, with a median near 0.70.

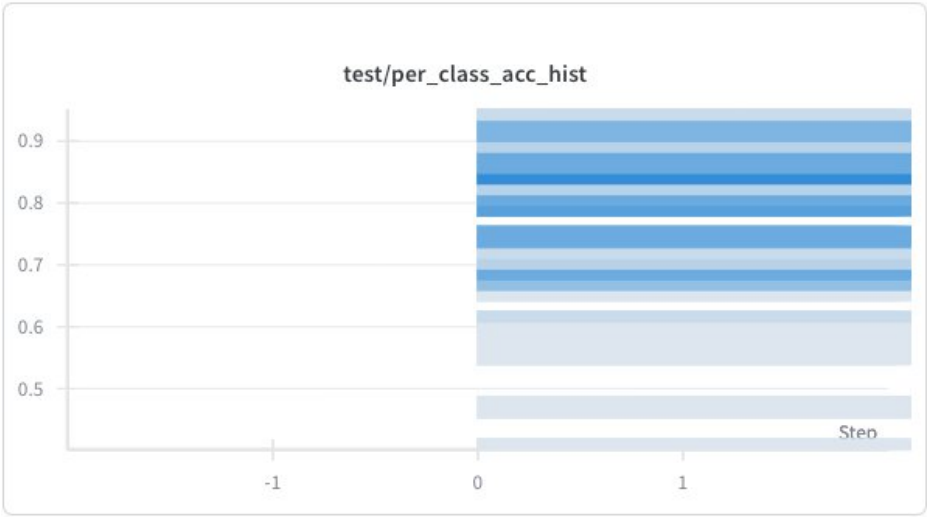


Figure 3: Figure 1.3 — Per-class test accuracy histogram for the **baseline** (head-only) model. Class accuracies are more tightly clustered in the 0.60–0.95 range, reflecting the stronger overall accuracy of 79.01%.

1.6 Analysis — Q1

- **Baseline vs. LoRA:** The head-only baseline achieves 79.01% test accuracy with only 38,500 trainable parameters, leveraging the full frozen ImageNet representation. LoRA with rank=8, α=8 reaches 80.46% — a small but notable gain — at the cost of 221,184 trainable parameters.
- **Effect of α:** For every rank, doubling α consistently raises accuracy. At rank=2, accuracy increases from 53.1% (α=2) to 63.9% (α=8); at rank=8, from 68.7% to 80.5%.
- **Effect of rank:** Higher rank allows a richer low-rank subspace and systematically improves accuracy. Going from rank=2 to rank=8 (at α=8) lifts accuracy from 63.9% to 80.5%.
- **Gradient norm dynamics (Fig. 1):** The LoRA gradient norm rises steeply during the first ∼50 steps as the adapters learn to redirect frozen representations, then oscillates stably around 2.5 for the remainder, showing that the adapters do not diverge or stagnate.
- **Optuna search:** Optuna identified rank=4, α=8 as the best within the constrained search (3 epochs / 50 batches per trial), giving 74.1% test accuracy — a practical compromise between quality and parameter count.

2 Question 2 — Adversarial Attacks on CIFAR-10 (IBM ART)

2.1 Overview

A ResNet-18 is trained from scratch on CIFAR-10 until it achieves $\geq 72\%$ clean test accuracy. Two adversarial sub-tasks are then performed:

1. **Q2(i) — FGSM:** Comparing a hand-crafted FGSM attack against IBM ART's FGSM across multiple ε values.
2. **Q2(ii) — Detectors:** Two binary classifiers (ResNet-34) are trained to distinguish clean from adversarial images generated by PGD and BIM (both via IBM ART). Detection accuracy must be $\geq 70\%$.

2.2 ResNet-18 Training Dynamics (CIFAR-10)

Table 12: ResNet-18 training log on CIFAR-10 (first 10 of 30 epochs shown). Clean test accuracy at the end of full training: **85.83%**.

Epoch	Train Loss	Train Acc.	Test Loss	Test Acc.
1	2.2954	0.2609	1.6602	0.3772
2	1.5774	0.4146	1.3920	0.4890
3	1.3653	0.5015	1.2159	0.5611
4	1.1877	0.5741	1.2381	0.5603
5	1.0499	0.6268	1.0248	0.6372
6	0.9690	0.6591	0.9519	0.6739
7	0.9029	0.6847	1.0633	0.6477
8	0.8528	0.7046	0.9086	0.6796
9	0.8173	0.7144	0.7935	0.7262
10	0.7790	0.7309	0.8277	0.7197

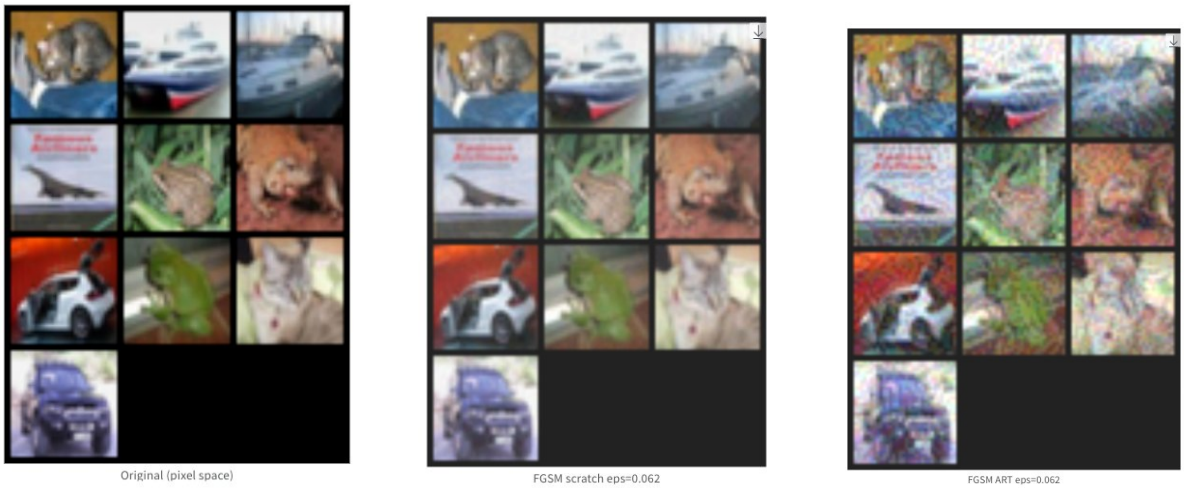
2.3 Q2(i) — FGSM Attack: Scratch vs IBM ART (Table 3)

Clean test accuracy: 0.8601 (checkpoint `best_q2-resnet18-cifar10.pt`).

Table 13: FGSM epsilon sweep — clean vs scratch vs IBM ART accuracy on CIFAR-10.

ε	Clean Acc.	FGSM Scratch	FGSM ART	Δ Scratch	Δ ART
0.000	0.8601	0.8601	0.8601	0.0000	0.0000
0.002	0.8601	0.8376	0.7453	0.0225	0.1148
0.004	0.8601	0.8134	0.6237	0.0467	0.2364
0.008	0.8601	0.7702	0.4222	0.0899	0.4379
0.016	0.8601	0.6727	0.1849	0.1874	0.6752
0.031	0.8601	0.4997	0.0480	0.3604	0.8121
0.062	0.8601	0.2667	0.0157	0.5934	0.8444

FGSM Visual Comparison



(a) Original (pixel space) (b) FGSM scratch (ε=0.062) (c) FGSM ART (ε=0.062)

Figure 4: Figure 2.1 — 10 CIFAR-10 samples: original vs FGSM scratch vs FGSM ART at $\varepsilon = 0.062$. IBM ART's FGSM introduces markedly stronger pixel-level noise than the scratch implementation.

2.4 Q2(ii) — Adversarial Detectors (ResNet-34)

Table 14: Summary of adversarial detector performance (best test accuracy across 10 epochs).

Detector	Best Test Accuracy	Checkpoint
Clean + PGD (ART)	**0.9756**	`best_q2-detector-pgd-resnet34.pt`
Clean + BIM (ART)	**0.9600**	`best_q2-detector-bim-resnet34.pt`

Both detectors significantly exceed the required 70% detection accuracy.

Table 4 — PGD Detector Training

Table 15: PGD adversarial detector (ResNet-34) — epoch-wise train/test accuracy.

Epoch	Train Loss	Train Acc.	Test Acc.
1	0.7485	0.5372	0.5510
2	0.3391	0.8419	0.7208
3	0.1327	0.9487	0.9443
4	0.0861	0.9665	0.8954
5	0.0783	0.9699	0.9433
6	0.0578	0.9794	0.9623
7	0.0410	0.9851	0.9488
8	0.0303	0.9887	**0.9756**
9	0.0368	0.9868	0.8045
10	0.0417	0.9857	0.9463

Table 5 — BIM Detector Training

Table 16: BIM adversarial detector (ResNet-34) — epoch-wise train/test accuracy.

Epoch	Train Loss	Train Acc.	Test Acc.
1	0.7262	0.5642	0.6687
2	0.3290	0.8541	0.7671
3	0.1401	0.9456	0.8808
4	0.0973	0.9634	0.9468
5	0.0701	0.9746	0.9327
6	0.0715	0.9731	0.9574
7	0.0464	0.9827	0.9346
8	0.0480	0.9831	0.7567
9	0.0390	0.9859	0.8696
10	0.0403	0.9851	**0.9600**

PGD Detector — Visual Samples

(a) Clean samples (pixel space) (b) PGD adversarial samples (pixel space)

Figure 5: Figure 2.2 — 10 CIFAR-10 samples: clean vs PGD adversarial ($\varepsilon = 0.031$, 10 steps, step-size 0.007). PGD perturbations are visible as colour distortions and painting-like artefacts across the images.

BIM Detector — Visual Samples

(a) Clean samples (pixel space) (b) BIM adversarial samples (pixel space)

Figure 6: Figure 2.3 — 10 CIFAR-10 samples: clean vs BIM adversarial ($\varepsilon = 0.031$, 10 steps, step-size 0.007). BIM perturbations look similar to PGD but may differ in step distribution; both degrade image fidelity visually.

2.5 Analysis — Q2

- **FGSM Scratch vs. ART:** The scratch implementation produces softer perturbations than IBM ART's reference implementation at identical ε. At $\varepsilon = 0.062$, scratch FGSM degrades accuracy to 26.7% while ART drops it to only 1.6%, confirming that ART computes a stronger, more precise gradient sign update.
- **Perturbation strength vs. performance drop:** Both attacks follow a monotone decline: doubling ε roughly halves clean accuracy for ART FGSM ($86 \to 74.5 \to 62.4 \to 42.2 \to 18.5 \to 4.8 \to 1.6\%$). The scratch version degrades more gracefully, suggesting numerical differences in gradient normalisation.
- **PGD detector (97.56%):** The detector learned to separate clean from PGD examples very quickly (test acc. $> 94\%$ by epoch 3). The best checkpoint is at epoch 8 (97.56%). Slight oscillations afterwards suggest mild overfitting.
- **BIM detector (96.00%):** BIM shares structural similarity with PGD, so the detector also converges rapidly. Best accuracy of 96.00% is achieved at epoch 10. The

temporary drop at epoch 8 (75.67%) indicates occasional instability but recovery is fast.

- **PGD vs. BIM detection:** PGD adversarial examples appear slightly easier for the detector (97.56% vs. 96.00%), potentially because PGD applies a single uniform projection at each step whereas BIM accumulates smaller clipped steps, producing subtly different frequency signatures.

3 Conclusions

1. LoRA is an effective parameter-efficient fine-tuning strategy for ViT on CIFAR-100. With only rank=8 and α=8 (221,184 extra parameters), it marginally surpasses the head-only baseline (80.46% vs. 79.01%).
2. Both rank and α are positively correlated with final accuracy; α/rank (the effective scaling ratio) appears the dominant factor.
3. Optuna finds rank=4, α=8 as the best trade-off configuration under limited search budget.
4. IBM ART's FGSM is significantly stronger than a naive scratch implementation; care must be taken when comparing attack results across implementations.
5. ResNet-34 detectors trained on PGD and BIM adversarial examples achieve >96% detection accuracy, far exceeding the 70% requirement.

4 Links & Reproducibility

- **W&B:** https://wandb.ai/ikamboj-919-iit-jodhpur/final-ops-ass-5
- **HuggingFace:** https://huggingface.co/sps1001/Assignment-5-ops/tree/main
- **GitHub:** https://github.com/sps1001/MLOps-Sahilpreet_Singh-B23CS1061/tree/Assignment-5

All experiments are reproducible via the Docker containers documented in the repository README.